

Design and Implementation of Gshare - Perceptron Hybrid Branch Predictor

Bhaskar Lade, Bhavana M Srinivasegowda, Sai Nikshitha Muthireddy, Suryavamshigoud Bolla

Department of Electrical and Computer Engineering

Portland State University

Abstract—This work is about implementation and execution of 5 branch predictors namely two-bit predictors, Gshare predictor, Perceptron predictor, tournament predictor and custom hybrid predictor which is the combination of Gshare and perceptron-based chooser. Worked on six different workload traces, the results clearly indicate that simple predictors struggle a lot with irregular branches, Gshare predictors work best with loop dominated code, perception works best with long history correlations, but it is very sensitive for floating point workloads. Both custom hybrid predictors and tournament predictors exhibited consistent performance compared to all others so far. The custom hybrid architecture got highest average accuracy and has smooth working in very severe misprediction spikes because of combination of correlation-based and learning based techniques.

Index Terms—Gshare, Perceptron, Tournament, MPKI, Custom, Accuracy, Miss Rate.

I. INTRODUCTION

Modern processors use deep pipelines and superscalar execution to achieve high instruction- level parallelism. Conditional branches have 20 to 25% of all executed instructions and give control hazards that may stall the pipeline for 10 to 25 cycles or more in case of misprediction. Early static prediction schemes like always taken for backward branches or hits which are provided by compilers, became insufficient since programs exhibited more dynamic behavior. This led to development of complicated dynamic branch predictors that learn from runtime patterns. The major steps in evolution have

- **Two-bit bimodal predictors:** track individual branch bias using saturating counters.
- **Gshare predictors:** incorporate global branch history to capture inter-branch correlations.
- **Perceptron predictors:** use neural-inspired learning to model long linear relationships in branch history.
- **Tournament predictors:** dynamically choose between local and global predictors based on accuracy.
- **Hybrid predictors:** combine correlation-based predictors like Gshare with learning-based models such as perceptrons.

Every generation updated the limitations of its predecessor like bimodal predictors miss the inter-branch correlation, Gshare cannot handle the linearly inseparable patterns, perceptron can only be used by high storage cost by certain workloads. No single predictor is perfect in everything but

modern designs like hybrid and tournament architecture are perfectly combined multiple mechanisms for best robustness and fewer mispredictions.

II. BACKGROUND AND RELATED WORK

Smith started working on two-bit saturating counters in 1980s from then branch predictions have undergone so much evolution. New innovations included global branch history, two level adaptive training, and hybrid designs made way to real processors like very popular Alpha 21264 and its successors. A major change was attained by perceptron predictor has a simple neural learning to capture long range correlations which is far better than earlier correlation-based prediction mechanisms. The recent work combines neural inspired predictors with traditional global history prediction mechanism which yield to effective mechanism. All the 5 predictors discussed in this paper represent a super trajectory of evolution of predictors.

III. IMPLEMENTATION

We have implemented all 5 predictors using common simulation framework and identical conditions for fair comparison. We have used 12-bit global history registers throughout 4096 entry tables for global predictor components and chooser in hybrid designs. The local history tables of tournament predictor have 512 entries for each branch history and pattern tables by using a 9-bit index. The perceptron predictor has 128 perceptrons which are indexed by 7 bits each weight 22 vector trained on 12 bit global history and additional bits to extend the effective history length. In this simulation all the predictors receive same trace file driven with program counter and outcomes a stream and they update their structures only on resolving branches and operate under same conditions and same resource assumptions.

A. Gshare Predictor

The Gshare Predictor has significant improvement in correlation awareness by combining the global branch history with indexing process. This predictor has a global history register with 10 to 14 bits wide that remembers the actual outcomes of the most recent branches. There will be 2-bit saturating counters in PHT but now the index into the table is framed by XOR of low bits of branch PC with current

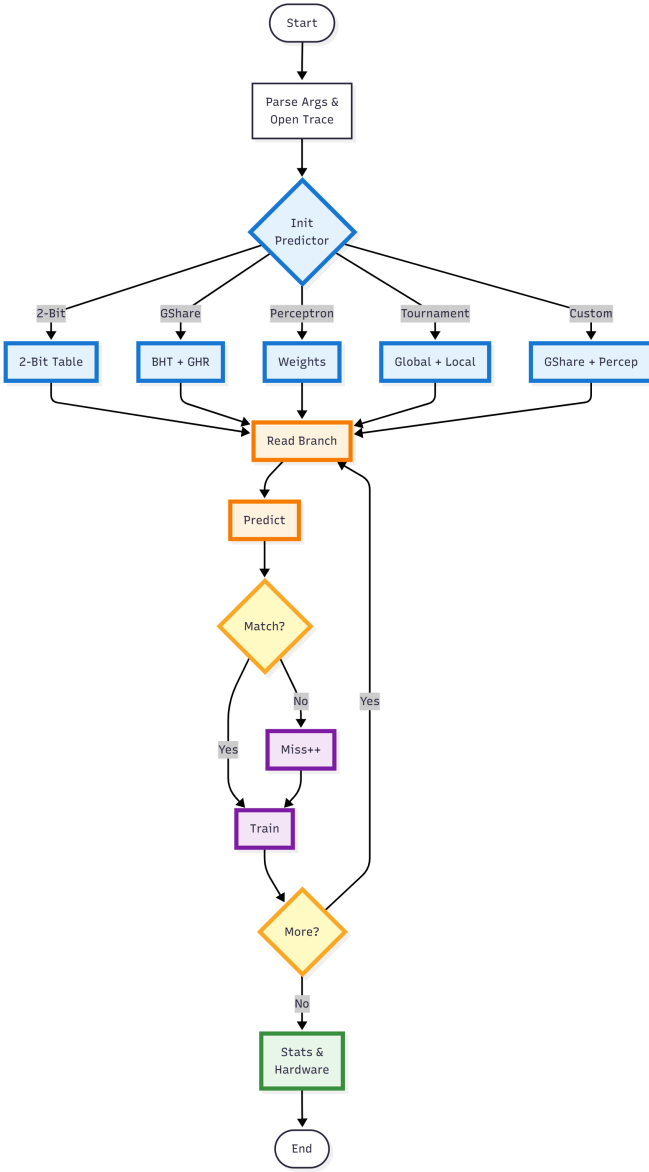


Figure 1. Implementation Flow

contents of GHR. This mix of XOR distribution correlated branches throughout the table in such a way that one counter can know the patterns connected with many preceding branches. The updated rules and predictions remain same as the bimodal case. The MSD is used to determine the guess, and the counter is updated as per actual outcome, but GHR is shifted left when new results are appended. Gshare perfectly works at regular, history-dependent structures of the if-else chains and nested loops. But as XOR is a linear operation of GHR, Gshare cannot know the patterns which are linearly cannot be separated in history space. With this discussion we can conclude that it is not reliable on more complex or irregular control flow.

B. Perceptron Predictor

This predictor has an extension to branch prediction with a lightweight neural learning mechanism which gives a chance to learn the long range, and linearly separable patterns that have traditional history-based schemes. It has a table for small perceptron with one for each indexed branch address. Each perception has a integer bias weight and vector of weights with 22 in implementation that match successive bits in the global branch history. We take +1 for taken and -1 for not taken for history bits. It computes the prediction by calculating the dot product of the weight vector with the signed history vector and adding in bias. The sign of the result indicates taken if nonnegative and not taken for negative for this prediction. The training happens only when the prediction is wrong or if the magnitude of output is below a small threshold value it means the predictor is not confident. In that case the weight is increased or decreased using simple perceptron learning rule with saturating values clamped from -128 to +127 as it is sign magnitude form. This method has more accuracy for deep and linear correlation workloads, but it requires much more storage compared to counter-based predictors, and it adds arithmetic delay in every prediction and update and It may get confused or will be difficult when branch behavior is nonlinear.

C. Custom Predictor

This predictor is a combination of fast and low overhead correlation Gshare tracking with the perceptron deep learning capacity for linear patterns. Both predictors will run parallel to keep fully trained on each branch outcome. The indexing is done by small chooser table for global history register and holding 2-bit saturating counters that decides The correct prediction as per lower counter favors the Gshare and higher counter values favor the perceptron. The updating of chooser value is only on certain branches which disagree or reinforce the correct. This design is more flexible to work as Gshare for short to medium length correlation and regular structures while allowing the perceptron working for longer or more complex dependencies dominate. With this we achieved better for worst case behavior. This has more storage compared to standalone predictor around 270 Kbits in our project that is driven by perceptron weight vectors of Ghsrae and modest overhead added by chooser.

D. 2-Bit Bimodal Predictor

The simple two-bit bimodal predictor works based on pattern history table, or PHT with 2^m entries where m is the number of bits is used to address it. Each entry contains a 2 bit counter whose value ranges from 0 to 3, with initial value of weakly taken or weakly not taken. This table is indexed by low order bits of program counter of the branch. Prediction is simple when high order bit of the counter is 1, The branch prediction is taken otherwise it is not taken. After branch is resolved, counter increases its value upon a taken

outcome, decremented upon not taken if there is saturation in the boundaries. This design has good performance for capturing persistent bias for individual branches like a loop back branch which is always taken but does not consider the correlation between different branches. The main advantage of this predictor is its very simple and low hardware cost and disadvantage is poor efficiency on interbranch dependencies workloads.

E. Tournament Predictor

This predictor became popular after Alpha21264. Tournament predictors are a competitive hybrid predictor that combines a local history predictor with a global history predictor in this work that is component of Gshare style and dynamic choosing of more reliable.

In our project the local predictor records a per branch history table with 9 bits indexed from the program counter to, in turn the index to a 2Nd table with 3-bit counters which are saturating. In this the operation of global predictors is same as traditional Gshare with its own pattern table. They both make predictions for each branch. The indexing of chooser table is done by global history register and this is made up of 2-bit saturating counters-selects to use higher values for local predictors and lower values for the global one. Always after the predictor is chosen, both global and local components are updated with the original outcomes. In this case the chooser is updated only after the two predictors do not agree with correct predictors value is increased and wrong predictors value is decreased. This simple working principle makes the tournament predictor adapt very fast to phases between the local or global correlations stronger areas. This makes it very effective and robust performance for widely varying workloads.

IV. SIMULATION METHODOLOGY

All the 5 predictors discussed above are tested by giving a trace driven simulator file that contains 6 different real world branch traces that vary from integer, floating point to very high irregular in control flow and loop incentive benchmarks. Each trace omits the branch pc sequence along with their original outcomes.

- The simulator performs operations for every dynamic branch.
- It presents the global history and feeds the program counter (PC) to the branch predictors.
- It waits for the original branch outcome.
- It increments the misprediction count when required.
- It updates all predictor structures exactly as hardware would.

It collects the important matrix that include prediction accuracy, rate of misprediction and mispredictions per kilo instructions (MPKI). It also records the tournament and hybrid custom predictors to how often each component is chosen by the chooser this is used to get more insight to workload specific

behavior. A full storage model has bit level costs that guarantee the simple predictors require less kilobits, but perception only consumes huge hundred kilobytes and hybrid stands between these extremes depending on consumption. Putting all together it allows good comparability of results with consistent cycles of accurate simulation framework.

V. RESULTS AND DISCUSSION

We evaluated five branch predictors: Gshare, Perceptron, Custom Hybrid (Gshare+Perceptron), Tournament, and 2-bit counter across six traces.

Input	Gsh	Perc	Cust	Tour	2Bit
fp ₁	99.0010	98.6618	98.8421	98.9675	98.6767
fp ₂	96.6373	79.6521	81.6823	92.0923	80.0638
int ₁	83.8089	84.0421	84.6140	84.4713	82.6594
int ₂	99.5037	99.3165	99.4140	99.2009	99.3039
mm ₁	92.1159	88.2970	91.8541	91.2064	88.5024
mm ₂	89.0640	88.0614	88.7725	89.4393	88.2954

Table I
ACCURACY COMPARISON OF PREDICTORS IN PERCENTAGE



Figure 2. Predictor type vs Accuracy graph

Table 1 presents the accuracy values for all predictors across six input traces. The results show that no single predictor dominates universally, but Gshare and the Custom predictor remain consistently strong across most workloads. In floating-point traces (fp₁ and fp₂), Gshare clearly outperforms Perceptron, while integer traces show closer performance among Gshare, Perceptron, and Custom. The Tournament predictor remains competitive but does not consistently achieve the highest accuracy. Overall, the table highlights that predictor effectiveness is heavily workload-dependent, especially between floating-point and integer-dominated traces.

Table 2 analyzes the total number of mispredictions made by each predictor. Lower values indicate better performance, and here Gshare again performs well in most cases, especially in fp₁, fp₂, and int₂. The Perceptron predictor, although powerful in theory, shows significantly higher mispredictions in several workloads such as fp₂, mm₁, and mm₂, indicating difficulty with certain branch patterns. The Custom

Input	Gsh	Perc	Cust	Tour	2Bit
fp ₁	15452	20700	17911	15970	20468
fp ₂	81446	492835	443663	191528	482864
int ₁	610680	601884	580314	585696	654036
int ₂	18638	25666	22008	30010	26142
mm ₁	237693	352829	245588	265114	346635
mm ₂	280387	306094	287862	270765	300094

Table II
MISPREDICTION COUNTS OF DIFFERENT PREDICTORS

predictor generally performs close to Gshare and often better than Perceptron. These results demonstrate that misprediction behavior varies significantly across trace types, suggesting that learning-based predictors may require better tuning for specific workloads.

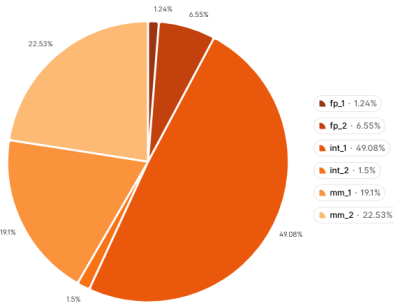


Figure 3. Miss Predictions graph

Input	Gsh	Perc	Cust	Tour	2Bit
fp ₁	9.989675	13.382493	11.579412	10.324500	13.232506
fp ₂	33.626900	203.478542	183.176724	79.076848	199.361780
int ₁	161.911203	159.579097	153.860185	155.287130	173.406294
int ₂	4.963099	6.834580	5.860494	7.991340	6.961333
mm ₁	78.840738	117.030366	81.459442	87.936050	114.975869
mm ₂	109.359697	119.386231	112.275181	105.606816	117.046044

Table III
MPKI COMPARISON ACROSS PREDICTORS

Table 3 reports the MPKI (mispredictions per thousand instructions), which reflects the efficiency of each predictor relative to total instruction count. The results closely mirror the misprediction trends: Gshare tends to provide the lowest MPKI for most traces, especially in fp₁ and fp₂. Perceptron shows very high MPKI in floating-point workloads, indicating poor generalization on those patterns. The Custom predictor remains competitive and often reduces MPKI noticeably compared to Perceptron. Tournament performs moderately well but does not surpass Gshare in most cases. MPKI results emphasize that predictor complexity does not always translate into practical performance gains.

Table 4 shows the selector (chooser) distribution for the hybrid design, indicating how often the system selected Gshare or Perceptron. In most traces, the chooser favors Perceptron, suggesting that it is identified as more accurate

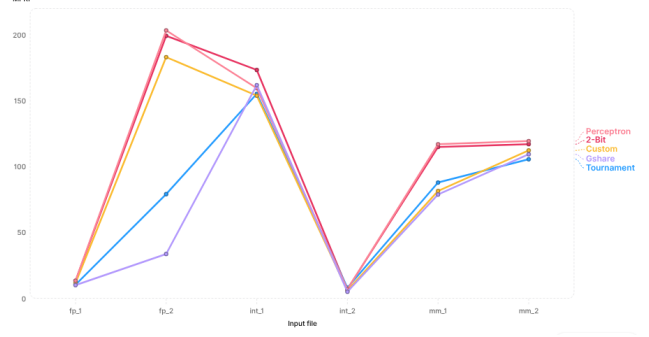


Figure 4. MPKI vs Predictor Type Graph

Input file	Gshare chosen	Perceptron chosen
fp_1	3.77	96.23
fp_2	41.59	58.41
int_1	27.72	72.28
int_2	4.07	95.93
mm_1	52.04	47.96
mm_2	31.52	68.48

Table IV
SELECTOR DISTRIBUTION IN PERCENTAGE

over time. However, in workloads such as mm₁ and fp₂, Gshare is selected more frequently, aligning with the earlier tables where Gshare outperformed Perceptron. This distribution highlights that hybrid predictors adapt to workload behavior, relying on Perceptron when its learning strengths are beneficial and switching to Gshare when simpler correlation-based prediction performs better. The table confirms that selector dynamics closely follow actual predictor performance trends.

Our evaluation shows that branch predictor performance varies significantly across workloads, and no single predictor dominates universally. Gshare performs well on traces with strong short-range global correlations, while Perceptron excels on irregular integer workloads but suffers large performance drops on certain floating-point traces. The Tournament and Custom Hybrid predictors provide the most stable behavior, maintaining low MPKI and avoiding the extreme accuracy losses seen in individual predictors. Chooser statistics confirm that the hybrid effectively adapts to branch characteristics by dynamically selecting between Gshare and perceptron. Overall, hybrid and tournament predictors offer the best balance of accuracy, robustness, and performance across diverse workloads, while simple 2-bit predictors are inadequate for modern branch patterns.

VI. CONCLUSION

This work implements and compares all predictors i.e two bit bimodal, Gshare, perception, Tournament and Custom hybrid by evolving the evolution of predictors from simple static to dynamic and complex predictors. We have located

the areas of strengths and weakness of each predictor with the help of simulation by a trace using common configuration for every predictor. We proved that hybrid custom predictors have highest and most reliable and consistent accuracy. There is close architecture used in modern predictors, and it has lot of scope for future experiments in this field that include TAGE.

VII. ACKNOWLEDGEMENT

We would like to thank our instructor, Yuchen Huang, for her valuable guidance and providing study resources in the class ECE 587 :Advance computer Architecture 1. Apart from her, we would also thank our Teaching assistant Sean Stalley for all his help and we took the help of many IEEE papers that provided valuable resources to this project.

REFERENCES

1. J. E. Smith, "A Study of Branch Prediction Strategies," ISCA, 1981.
2. S. McFarling, "Combining Branch Predictors," DEC WRL Technical Report, 1993.
3. Daniel A. Jiménez and Calvin Lin, "Dynamic Branch Prediction Using Neural Networks," HPCA, 2001.
4. S. Pan, K. So, and J. Rahmeh, "Improving the Accuracy of Dynamic Branch Prediction Using Branch Correlation," ASPLOS, 1992.
5. T.-Y. Yeh and Y. Patt, "Two-Level Adaptive Branch Prediction," MICRO, 1991.
6. J. Stark et al., "On Predicting the Future with 1.75 Bits of Memory," HPCA, 1998.